

CortexAI Ø>Ýà&j

Recursive Complexity-Based Dynamic Multi-Agent Architecture & Algorithm Specification

CortexAI is an enterprise-grade adaptive multi-agent AI system that evaluates task complexity, dynamically creates hierarchical Directed Acyclic Graphs (DAG), recursively decomposes complex subtasks into atomic units, selects the cheapest capable agent for each leaf task, executes them with shared incremental working memory, calculates measurable trust, selectively escalates low-trust results, and synthesizes structured executive outputs.

1. Microservices Architecture & Topology

- API Gateway (Port 8000): Central reverse proxy with dual-session authentication (HttpOnly cookie + Authorization Bearer header), CORS management, request routing, and SSE multiplexing.
- Auth Service (Port 8001): Firebase token verification, MongoDB user persistence, and Redis distributed session caching.
- Chat Service (Port 8002): Conversation threads, message history persistence, and chat telemetry.
- Agent Service (Port 8003): Recursive task decomposition, complexity evaluation, dynamic agent selection, DAG execution, RAG retrieval, PPT/PDF synthesis, and observability.
- Billing Service (Port 8004): Razorpay payment orders, webhook verification, credit deduction, and subscription quotas.

2. Complete Inventory of Implemented Algorithms

Algorithm 1: Hybrid Complexity Evaluation Engine

Calculates objective complexity (0 - 100) using the formula: $ComplexityScore = 0.30D + 0.20S + 0.20R + 0.15C + 0.15U$

- D (Dependency): Computed from graph in-degrees and recursion depth: $(depCount * 30) + (depth * 15)$, clamped to 10 - 100.
- S (Reasoning Steps): Step count normalized to 0 - 100 (1 step=20, 2 steps=35, 3-4 steps=55, 5-6 steps=75, 7+ steps=95).
- R (Retrieval): Tool signals (live search=95, PDF RAG=90, multi-source search keywords=85, internal knowledge=15).
- C (Computation): Full-stack code=95, standard code=80, math formulas & algorithm simulation=85, basic transformation=15.
- U (Uncertainty): Multi-hypothesis / strategic analysis=75, exploratory inquiry=55, clearly specified task=20.
- Classifications: 0 - 39: EASY | 40 - 69: MEDIUM | 70 - 100: COMPLEX.

Algorithm 2: Recursive Task Decomposer with Stop Conditions

Recursively splits complex subtasks (Complexity >= 70) into 2-3 focused child subtasks, evaluating each child independently. Enforces strict stop conditions:

1. Stop if depth >= MAX_DECOMPOSITION_DEPTH (default 3)
 2. Stop if ComplexityScore < 70 (EASY or MEDIUM)
 3. Stop if task is already atomic/actionable (length < 40 chars & no conjunctions)
 4. Stop if decomposition overhead exceeds expected benefit

Algorithm 3: Cheapest Capable Dynamic Agent Selection

Selects the lowest-cost agent capable of meeting task complexity and quality requirements using multi-factor scoring:

$AgentScore = w_cap * CapabilityMatch + w_qual * Quality + w_suit * Suitability + w_tool * ToolCompat - w_cost * CostPenalty$

- Explainability: Returns selectedAgent, calculated score, human-readable reason, and list of alternative candidates evaluated.

Algorithm 4: Hierarchical Execution Tree & Topological Dispatch

Maintains a tree data structure tracking nodes, depths, factor scores, models, and trust metrics. Extracts only leaf nodes (isLeaf=true) for concurrent DAG execution based on completed prerequisite dependencies.

Algorithm 5: Shared Incremental Working Memory

Provides tasks with scoped prerequisite context: passes only direct dependency outputs and newly generated delta facts rather than duplicating the entire conversation history.

Algorithm 6: Measurable Multi-Domain Trust Evaluator

Calculates objective trust scores (0 - 100) from domain evidence rather than arbitrary model self-confidence:

- Coding: Valid multi-file artifacts (+35), code blocks (+25), missing code penalty (-25), error traces (-30).
- Research / RAG: Verified citations (+8/source up to +35), citation tags (+25), depth (+20).
- Analysis / Math: Markdown comparison tables (+30), quantitative numbers (+15), headings (+10).
- Classification: 85 - 100: HIGH | 60 - 84: MEDIUM | 0 - 59: LOW (triggers escalation).

Algorithm 7: Targeted Single-Leaf Escalation Manager

When a leaf task produces Low Trust (< 60), selectively upgrades the assigned model to a stronger capable tier and re-runs only the affected subtask, updating shared memory without regenerating the full workflow (max 2 attempts).

Algorithm 8: Fine-Grained Cost & Usage Telemetry Tracker

Tracks estimated vs actual costs across all tasks, recursion levels, and escalations, aggregating totalTokens, duration, and averageTrust.

Algorithm 9: Distributed Real-Time Cancellation Lifecycle

Dual-layer cancellation (in-memory + Redis key with 1h TTL) broadcasting workflow_cancelled events and checking assertNotCancelled() before and after every async call.

Algorithm 10: PDF RAG Pipeline

pdf-parse extraction, RecursiveCharacterTextSplitter (1000-char chunks, 200 overlap), Qdrant vector search (k=5) with tokenized keyword fallback.

Algorithm 11: Web Search Normalization & Citation Ranking

Sanitizes HTML tags and raw query JSON, deduplicates URLs, limits snippets to 400 characters, and renders markdown citation references.

Algorithm 12: Executive Map-Reduce Synthesizer

Aggregates diverse subtask findings into an executive report with clean comparison tables, thematic headings, strategic takeaways, and download links.

Algorithm 13: Dynamic Presentation & PDF Engines

Generates styled 16:9 widescreen PowerPoint decks (pptxgenjs) and styled PDF documents (pdfkit).

Algorithm 14: Dual-Session Authentication

Validates session tokens across cookies, Bearer headers, and x-session-id against Redis cache.

Algorithm 15: Real-Time SSE Tree Streaming

Streams live execution tree updates (tree_initialized, agent_started, agent_completed, node_escalation) over Server-Sent Events.

